

Universidad del Quindío
Ingeniería de Sistemas y
Computación
Programación III

Proyecto final:
Sistema Distribuido de Gestión
Inmobiliaria en Elixir

1. Definición del Proyecto final

1.1 Equipo de trabajo

Indicar en la tabla los participantes del proyecto.

1.2 Enunciado del proyecto

Se debe implementar un sistema multiusuario que simula el funcionamiento de una inmobiliaria virtual en una ciudad o región. Varios usuarios se conectan en tiempo real desde la línea de comandos (CLI) y pueden interactuar como clientes, vendedores o arrendadores dentro de la plataforma.

Los vendedores y arrendadores pueden publicar propiedades de diferentes tipos, tales como casas, apartamentos, oficinas o lotes. Los clientes pueden consultar propiedades disponibles, filtrarlas, comprarlas, arrendarlas o comunicarse con el responsable de la publicación.

Cada propiedad publicada será administrada como un proceso independiente, permitiendo manejar múltiples propiedades, solicitudes y negociaciones en paralelo. El sistema debe gestionar usuarios, publicaciones, mensajes, operaciones y rankings de actividad.

El objetivo es simular el flujo real de una inmobiliaria, aplicando concurrencia, supervisión, persistencia y comunicación entre procesos en Elixir.

1.3 Objetivos de Aprendizaje

- Aplicar concurrencia y procesos en Elixir.
- Implementar un sistema distribuido usando GenServer y Supervisores dinámicos.
- Manejar persistencia en archivos de texto plano.
- Construir un sistema interactivo en tiempo real con usuarios, propiedades, operaciones y mensajería.
- Modelar un dominio más complejo con distintos roles y tipos de transacción dentro de una inmobiliaria

1.4 Funcionalidades Requeridas

1.4.1. Gestión de Usuarios

El sistema debe permitir el manejo de usuarios con diferentes roles dentro de la inmobiliaria.

- Registro y login de usuarios.
- Un usuario puede tener rol:
 - cliente
 - vendedor
 - arrendador
- Los usuarios se almacenan en users.dat con la siguiente información:
 - nombre de usuario
 - rol
 - contraseña
 - puntaje acumulado
 - Cada usuario puede consultar su puntaje.
- El sistema debe mostrar un ranking global de usuarios más activos.

1.4.2. Conexión

- Conectarse (connect username password).
- Si no existe → registro automático.
- Desconectarse (disconnect).

Reglas:

- Si el usuario no existe, se registra automáticamente.
- Si ya existe, debe validarse la contraseña.
- Un usuario autenticado puede ejecutar acciones según su rol.

1.4.3. Publicación de Propiedades

Los vendedores y arrendadores pueden publicar propiedades dentro del sistema.

Cada propiedad debe registrar, como mínimo:

- id de propiedad
- tipo de propiedad (casa, apartamento, oficina, lote, etc.)
- modalidad (venta o arriendo)
- ubicación
- precio
- número de habitaciones
- área
- estado (disponible, reservada, vendida, arrendada)
- propietario o publicador

1.4.4. Consulta de Propiedades

Los clientes pueden listar y consultar las propiedades disponibles en la plataforma, según el tipo o todas.

El sistema debe permitir búsquedas básicas por:

- tipo
- modalidad
- ubicación
- rango de precio
- estado disponible

1.4.5. Operaciones Inmobiliarias

Los clientes pueden realizar solicitudes sobre propiedades disponibles.

- **Comprar una propiedad:** Si la propiedad está en venta:
- **Arrendar una propiedad:** Si la propiedad está en arriendo

Reglas:

- Una propiedad solo puede ser comprada o arrendada si está disponible.
- Al confirmarse una operación:
 - el cliente obtiene puntos
 - el vendedor o arrendador obtiene puntos
 - la propiedad cambia de estado
- Si dos clientes intentan operar simultáneamente sobre la misma propiedad, solo una operación debe completarse.

Esto obliga al sistema a manejar concurrencia adecuadamente.

1.4.6 Comunicación entre Usuarios

El sistema debe permitir mensajería entre clientes y responsables de propiedades.

Reglas:

- Un cliente puede enviar mensajes al publicador de una propiedad.
- El vendedor o arrendador puede revisar los mensajes asociados a sus propiedades.
- El historial debe quedar persistido.

1.4.7 Resultados y Ranking

El sistema debe registrar todas las operaciones inmobiliarias en un archivo de historial.

Archivo: results.log

Cada registro debe guardar:

- fecha
- cliente
- vendedor o arrendador
- id de propiedad
- tipo de operación (compra o arriendo)
- ubicación
- precio
- estado final

Ejemplo:

2026-03-17; cliente=ana; responsable=luis; propiedad=prop123;
operacion=compra; ubicacion=Armenia; precio=250000000; status=Completada

Además:

- Se actualizan los puntajes de los usuarios en users.dat.
- Se muestra ranking global de actividad.

- Puede haber ranking por compradores, vendedores y arrendadores.

1.4.8 Persistencia

El sistema debe usar archivos de texto plano para conservar la información.

Archivos sugeridos:

- users.dat → usuarios, roles, contraseñas y puntajes
- properties.dat → propiedades registradas
- results.log → historial de compras y arriendos
- messages.log → mensajes entre usuarios
- locations.dat → lista de ubicaciones válidas

1.4.9 Concurrencia y Tiempo Real

El sistema debe soportar:

- múltiples usuarios conectados simultáneamente
- varias propiedades activas al mismo tiempo
- múltiples solicitudes de compra o arriendo en paralelo
- mensajería concurrente
- supervisión de procesos de propiedades

Cada propiedad debe ser un proceso independiente para permitir que varias operaciones ocurran al mismo tiempo sin afectar las demás.

1.5 Flujo Ejemplo

1.5 Flujo Ejemplo

1. **Carlos** se conecta como vendedor:
connect carlos 1234 vendedor

2. Carlos publica una casa en venta:
publish_property tipo=casa modalidad=venta ubicacion=Armenia precio=300000000
habitaciones=4 area=180

3. **Ana** se conecta como cliente:

connect ana 4321 cliente

4. Ana consulta propiedades:

list_properties

5. Ana encuentra la propiedad publicada por Carlos y envía un mensaje:
send_message prop001 Hola, me interesa esta casa. ¿Sigue disponible?

6. Ana decide comprarla:
buy_property prop001

7. El sistema valida que la propiedad sigue disponible y registra la operación.

8. Al completarse:

- Ana obtiene +10 puntos.
- Carlos obtiene +15 puntos.
- La propiedad cambia a estado vendida.

9. Se guarda en `results.log`:

2026-03-17; cliente=ana; responsable=carlos; propiedad=prop001; operacion=compra;
ubicacion=Armenia; pre

1.6 Archivos Recomendados

proyecto_inmobiliaria/

| — lib/

| | — inmobiliaria/

| | | — server.ex # Servidor principal y comandos

| | | — property.ex # GenServer de cada propiedad

```
| | └─ supervisor.ex # DynamicSupervisor de propiedades
| | └─ user_manager.ex # Gestión de usuarios
| | └─ property_manager.ex # Registro y consulta de propiedades
| | └─ message_manager.ex # Gestión de mensajes
| | └─ location.ex # Validación de ubicaciones
| └─ data/
|   └─ users.dat
|   └─ properties.dat
|   └─ results.log
|   └─ messages.log
|   └─ locations.dat
| └─ test/
| └─ mix.exs
```

1.7 Posibles Extensiones

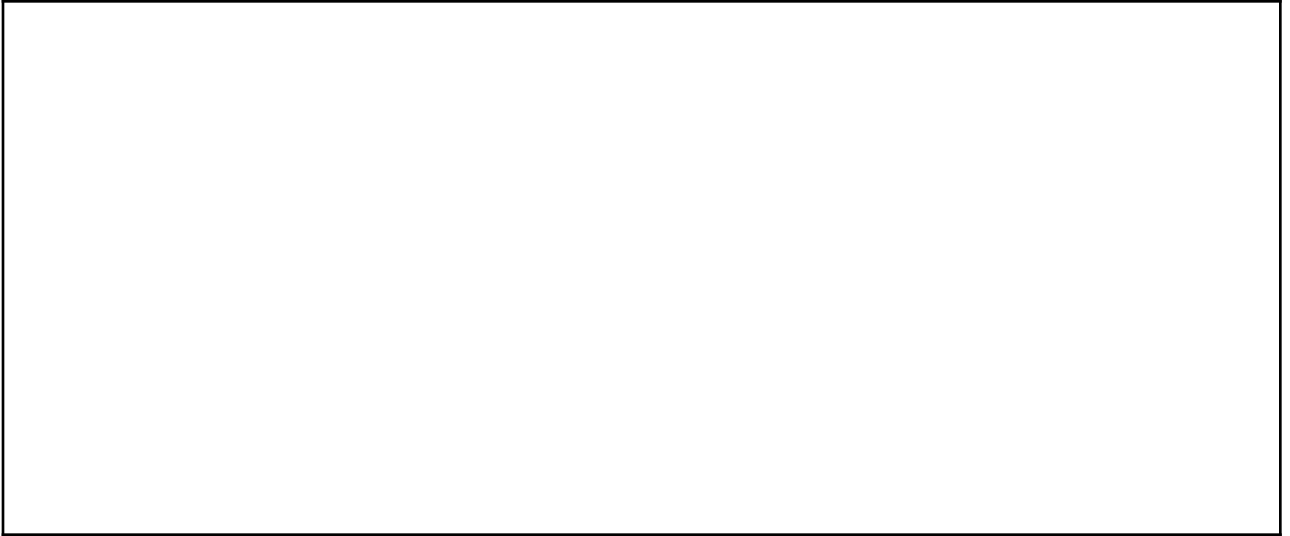
- Ranking separado por compradores, vendedores y arrendadores.
- Reserva temporal de propiedades.
- Cancelación de operaciones con penalización.
- Soporte para imágenes o descripciones más detalladas.
- Persistencia en JSON.
- Chat en tiempo real entre cliente y vendedor.
- Soporte distribuido con nodos distintos de Elixir.
- Recomendación de propiedades según preferencias del cliente.

2. Desarrollo del Proyecto final

En las siguientes secciones se muestra el desarrollo del proyecto, complete la información solicitada.

2.1. Diagrama de Arquitectura de Módulos

Colocar una imagen del modelo de datos que represente el enunciado.



1.6 Modelo de datos

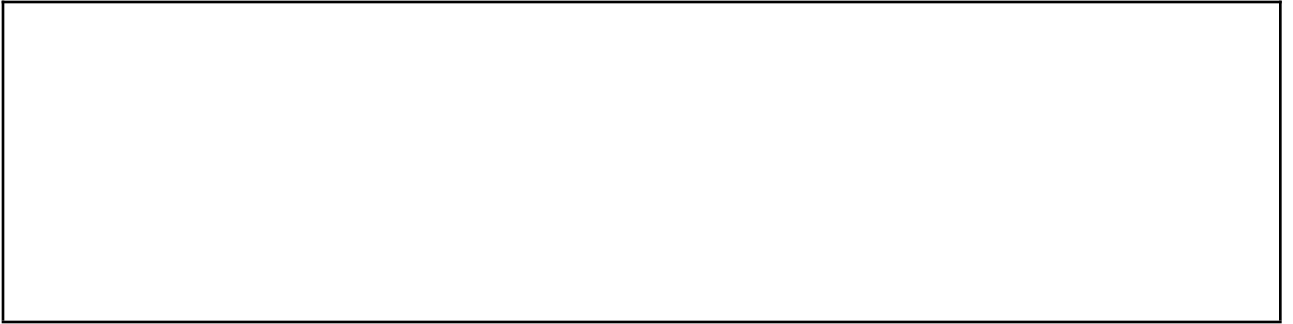
Colocar una imagen del modelo de datos que represente el enunciado.



1.7 Diagrama de Secuencia



1.8 Diagrama de Procesos y Supervisión (OTP Tree)



3. Estructuración del proyecto

En las siguientes secciones cumplir con lo que se solicita y colocar una imagen de acuerdo a la sección.

2.1 Estructura del proyecto

Utilizar la estructura de trabajo.



4. Demostración

4.1 Video de demostración

Realizar un vídeo haciendo una demostración de la aplicación desarrollada cumpliendo con lo solicitado.



5. Repositorio

5.1 Link repositorio

Entregar el link del repositorio.

